

The Substrate Vision — a mark-to-market enumeration

Status: living. Re-scored against the ecosystem periodically (see [Mark-to-market protocol](#)). First scored 2026-07-17.

The pitch

I'm trying to build a completely consistent computing substrate that uses PureScript for the lingua franca everywhere — across multiple runtimes to form a kind of "Lisp machine for the 21stC" except with strong typing, or a Unix-style system but with top-to-bottom consistency. The goal is to leverage all sorts of existing powerful sub-systems as appropriate — operating systems, DBs, file-systems (incl Nix) — but present a completely consistent and powerful layer across it, where even a small "shell script" can leverage `Data.Set` or `traverse`, and where all the non-essential complexity has been boiled out of it, not unlike training an LLM. — AFC, 2026-07-17

Clarification (same conversation): the goal is **not** to replace every Python script with PureScript. It is to reach the universality Unix shells get through `argv` + sockets + file descriptors — the thing that makes it possible to write a script that gathers information at the command line, forks a compiler and a linker, prints to the terminal, writes a log file. But without the cost: as soon as you leave process in Unix you drop to the common denominator (byte streams, ad-hoc exit codes) and you are in a world of emergent complexity. The challenge is to get to the level where you **always have the choice**, through eDSLs, to combine computations with principled, strongly-typed connections — whether it's system-level work or a database query in a user app.

The crux: mandatory bytes vs. optional types

Unix's universality comes from a **tiny mandatory common denominator**: every process, no matter its language, must speak `argv`, environment, file descriptors, exit codes, signals. Because it is mandatory, everything composes; because it is bytes, every composition point erases types and re-imposes parsing, exit-code folklore, and stringly errors.

The substrate's answer is not a new mandatory layer (that is the Lisp machine's isolation failure — a wall instead of a floor). It is a small set of **typed carriers**: ways of crossing each kind of boundary that are *everywhere available but never compulsory*. Two clauses govern every carrier:

1. **The choice clause.** At any boundary — in-process, cross-process, cross-machine, cross-runtime — there exists a typed carrier you can reach for instead of raw bytes, and it is *cheap enough* that reaching for it is the default, not a virtue.
2. **The degradation clause.** Every typed carrier degrades gracefully to Unix. Its wire form is inspectable bytes (JSON on an fd, OSC on a socket); you can still pipe it to `grep`. Universality through choice, not through walls.
3. **The openness clause.** The substrate leans hard on existing open technology — OS, Nix, Git, JSON, SQL, open wire protocols — and explicitly refuses to lean on commercial cloud providers whose protocols are not open. GitHub is tolerated for now (git itself is the open substrate; GitHub is

a replaceable convenience). Dropbox and its kind are absolutely not. A carrier or part that can only be reached through a closed protocol is not a substrate part.

4. **The typed-boundary principle — parse don't validate, and MISU, plumbed to the bottom.** At every carrier, incoming data is parsed into types that cannot represent illegal states — never validated and passed along as strings-with-vibes. This is the substrate-wide generalization of the code-level idioms: a Snapshot that is *total* (every path **Exists** xor **Missing** — jtms-make's soundness lever), Bosun's typed compose specs, the SysEx models, the contract modules. Unrepresentable illegal states are the substrate's answer to whole categories of Unix defensive plumbing (the **set -euo pipefail** genre exists because the shell validates nothing).

The complexity wager

An aspiration rather than a directive: **for every way that making PureScript the shell/make/kubernetes lingua franca ADDS to the complexity budget, we aspire to typeclasses and laws REDUCING it.**

Typed shell scripting will sometimes be a PITA relative to bash; the wager is that the ledger nets positive because folds, applicatives, alternatives, and traversals are *right there in the language* — no awk sublanguage, no jq sublanguage, no cron syntax, no Helm templating, each with its own quoting rules and no laws. Every sublanguage Unix accreted is complexity the typed substrate pays off with one language plus instances. Part III is this wager's account book: each lawful instance found or imported is a credit against the typing tax.

The lingua franca makes one move possible that Unix never had: when both ends of a boundary are in-family, **the contract is a shared PureScript module** — no IDL, no protobuf, no schema drift. The type is the protocol. The ecosystem has already discovered this independently at least six times (see B2); this document's job is to make such discoveries convergent instead of repeated.

An open edge on that move: some ends are *deliberately* foreign — the hardware daemons are Rust because Rust is the appropriate language there. How a foreign-implementation end participates in a shared-type contract (generated bindings? codec-validated at the boundary per B1? conformance tests against the PureScript contract module?) is an open question tracked at B5.

The jointing principle

Joints belong at boundaries that already exist — process, machine, runtime, trust — never at boundaries invented for a single project's convenience. For each boundary kind this document names the blessed carrier. The working rules:

- Both ends in-family → **contract-as-shared-module** (B2).
- Far end foreign (browser API, external service, hardware) → **codec at the boundary** (B1); parse, don't validate; never trust a foreign return type.
- Realtime/musical → the realtime carriers (B7); timestamps and anchors, not "soon".
- A new project must be able to say *which carriers it uses* by pointing at this enumeration. If none fits, that is an amendment to this document — a named new part — not an ad-hoc improvisation.

Part I — The enumeration: integrating what exists

A calibration note: the current evidence base over-represents the music rig, because that is where the most boundary-crossing work has happened so far. Expect the examples (and the extraction pressure) to

rebalance as use-cases grow; the music instances remain valid evidence of each carrier, just not its whole story.

Status vocabulary (used in both parts): **SETTLED** (done, boring, relied on) · **EXISTS** (working, may still move) · **PARTIAL** (convention or fragment, not yet dependable everywhere) · **AD HOC xn** (the need is proven by n independent hand-rolled instances; extraction wanted) · **PLANNED** (agreed direction, nothing built) · **BLUESKY** (survived the sift, no commitment yet) · **MISSING** (enumerated because the vision requires it; nothing exists).

A. The surface — lingua franca and its runtimes

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
A1	Compiler, registry, spago	cc + libc	SETTLED (upstream)	
A2a	JS/Node runtime	the default ABI	SETTLED	everything web-facing
A2b	BEAM runtime (purerl)	— (Unix has no supervised runtime)	EXISTS	purerl-tidal: supervisor trees, hot-load
A2c	Julia backend (Jurist)	—	EXISTS	reference architecture for the family
A2d	Python backend (Pythia)	—	EXISTS	422/426 byte-identical on shared corpus
A2e	Go backend (Gnomon)	—	EXISTS	any-runtime CoreFn→Go
A2f	Lua backend	—	PARTIAL	present in purescript-backends; maturity unassessed
A3	The idiom canon	POSIX + man-page conventions	EXISTS	Elements of PureScript Style (178 entries) + /purescript-style , /fp-police skills. Consistency is compression: one idiom means a model needs only the <i>facts</i> , the idiom is already in training data
A4a	Declaration index	apropos / whatis	EXISTS	API-INDEX.md, 2,157 modules / 30,968 decls, make api-index
A4b	Semantic (embeddings) lookup	—	PLANNED	occasional pass over the same harvest; "better than grep"; likely an MCP

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
A4c	Code cartography	—	EXISTS	Minard (DuckDB ground truth + visual front end)
A4d	Human-legible typography	troff/man	EXISTS	Specimen, the-prelude

B. Typed carriers — the boundary discipline

This is the crux layer: what replaces argv, fds, exit codes.

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
B1	Canonical encoding + codec discipline	bytes	PARTIAL	codec-argonaut convention; ~193 codec mentions across 2,157 indexed modules — real but not universal
B2	Contract-as-shared-module protocols	— (Unix cannot do this)	AD HOC x6	TidalProtocol.{Binding,Message,Registry}; Reef.Protocol + Reef. {Balistes,Stellatus,Vetula}.Protocol; Triggerfish.Selene.Wire; Atlas.Protocol (Jurist example). Extraction wanted: a named pattern/library — envelope, versioning, error channel — distilled from the six
B3	Typed error channel across boundaries	exit codes + errno	MISSING	today: exit ints and strings. Wanted: a convention (Variant-or-ADT over the wire) that every carrier shares, so failure composes like success does
B4	Typed process invocation	fork/exec + \$?	AD HOC x3	spawnDuckDB (Minard), execFile, spawnFromParentWithStdin — each a local FFI. Wanted: one library — typed argv/env/cwd, captured stdout/stderr, exit as ADT. This is the "forks a compiler and a linker" case
B5	Typed local services (control sockets, daemons)	unix domain sockets + daemontools	AD HOC x3	es9-daemon ~/es9/control.sock (Selene stack), fh2 daemon ~/fh2/control.sock , and Bosun already <i>models</i> UnixSocket AbsPath in its Exposure type. Two extractions wanted: (a) the shared server/client library; (b) a standardized daemon-creation process — our daemons are mostly hardware-specific and written in Rust, which is the right language, but how a foreign-

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
				language daemon fits the typed substrate is an open question (see the crux). Something to look at deliberately, not case-by-case
B6	Typed HTTP	inetd + CGI	PARTIAL	HTTPurple is a KEY technology viewed through this lens — it is the substrate's default in-family service carrier, already deployed throughout (Marginalia, Calypso, chair-server). What's missing is the second half: routes-as-shared-types and a blessed typed client, so a service's route table becomes a B2-style contract module
B7	Realtime carriers (OSC, MIDI, SysEx, Link)	—	EXISTS	es9-config/fh2-config (typed SysEx models + DSLs), link-spike (Link→OSC anchor fan-out), es9-daemon. The music rig is the substrate's proving ground for hard-realtime typed boundaries

C. The shell — scripting layer

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
C1	Script runner	<code>#!/bin/sh</code>	MISSING	a single-file PureScript script with declared deps and tolerable start latency. Today the entry cost is a spago project. Until this exists, "even a small shell script gets <code>traverse</code> " is aspiration, not fact
C2	Location & path addressing	coreutils + the filesystem namespace	PARTIAL	node-fs wrappers per project today. Two threads: (a) <code>pathy</code> exists in the registry (type-safe paths) — evaluate it rather than rebuild; (b) the deeper question is bigger than paths: addressing things by location is substrate-critical , and we should take best practice from commercial and research distributed systems (URIs as the general form). A substrate-layer ADT is the likely start — something like <code>data AssetLocation = Uri ... LocalPath</code> — one type every carrier addresses assets through, with the filesystem as just one constructor
C3	The pipeline eDSL	the pipe	MISSING	the crown-jewel gap. One eDSL in which a stage is a pure function, an Aff, or an external process (via B4), and every joint is a codec (B1) or shared type (B2) rather than byte-hope. This is the precise

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
				meaning of "always have the choice": the scenario "gather info at the CLI, fork a compiler, print to terminal, write a log" expressed as one typed composition

D. Data layer

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
D1	Embedded-DB access (DuckDB/SQLite)	flat files + awk	AD HOC x3	Minard (DuckDB FFI), Marginalia (DuckDB), larder (SQLite-adjacent). Same Effect (Promise a)/toAffE shape re-rolled per project. Direction: abstract away from the specific persistence substrate where appropriate <i>and performant</i> — the abstraction must never cost the performance that made DuckDB the choice
D2	Typed query discipline	—	PARTIAL	raw SQL strings + codecs on results today. Direction: leverage the rowtype-yoga SQL work — queries validated against schemas at compile time via row types — and integrate that with data-integrity and schema-analysis tooling (minard-db). SQL itself stays (a fine satisfice); what gets typed is the contract between query text, schema, and result row
D3	The personal larder	\$HOME	EXISTS	infovore-larder-db (~130k photos, music, books, quotebook). Explicitly a personal project with no claim to "right architecture" — draw only small lessons from it, chiefly about how the personal archive nearly everyone now has should be managed. It is, however, the motivating case for the openness clause (crux, clause 3): the archive must never depend on a closed commercial protocol

E. Orchestration, supervision, time

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
E1	Supervision + lazy-spawn (Bosun)	init/systemd	EXISTS	typed PureScript compose specs; bosun serve + bosun supervise ; Chair as the operational surface
E2	In-runtime supervision	—	EXISTS	BEAM trees via purerl (purerl-tidal per-voice supervisors). Open question: is the BEAM a special class of "container" from Bosun's point of view? Bosun should extend its management — or at minimum its <i>visibility</i> — into containers and into the BEAM, so one operational surface sees OS processes, Docker containers, and BEAM supervision trees
E3	Shared clock	cron (feeble analogue)	EXISTS	Ableton Link → link-spike → /link/anchor fan-out to purerl-tidal + es9-daemon. A substrate with a <i>musical</i> clock — tempo-relative time as a first-class service — is a genuine novelty worth naming
E4	Reproducible provisioning	tarballs + prayer	PLANNED	Policy: use Nix until we can rewrite Nix . PureNix (a PureScript→Nix backend) exists and appears maintained/in use — fitness for our purpose unassessed. Quartermaster (Marginalia #238) is the project: leverage whatever it can, shim between systems, and offer one comprehensive provisioning surface. Acceptance bar deliberately low at first: it is fine for Quartermaster to be very weak as long as its errors are clear. Today the substrate reproduces by convention (CLAUDE.md + Makefiles + registry pins)

F. Interfaces & delivery

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
F1	UI substrate (web vessel)	curses/X	SETTLED	Halogen + Hylograph/HATS; 15+ published libs. Settled <i>for the web vessel</i> — see F4
F2	Static delivery	—	EXISTS	cloudflare-sites + per-project make dist

#	Part	Unix analogue	Status (2026-07-17)	Evidence / notes
F3	Service delivery	—	EXISTS	polyglot-deploy (Docker profiles) + Bosun-native
F4	Native UI vessel	AppKit/Cocoa	BLUESKY	Evaluation project: AppKit/SwiftUI vs web frameworks vs Halogen — what would a fully-FP, fully-native UI library look like as a substrate tenant? See below

F4 — the native-vessel question (bluesky, 2026-07-18)

Every substrate UI today rides a browser engine, so the most complexity-laden artifact in computing — the web platform — sits inside the loop of every projection. The captured idea: **a project to evaluate interface-building technologies** (Swift's AppKit/SwiftUI, the web frameworks, Halogen), compare and contrast, and ask what possibilities exist for a fully-FP, fully-native UI library sitting directly on the substrate.

Why this is substrate-shaped and not just "try SwiftUI":

- **The backends are the thesis** (section A): Jurist/Pythia/Gnomon prove the surface retargets. A native vessel is *one more runtime* — PureScript compiled or FFI-bridged onto a native toolkit — not a second language. The evaluation should price that path against the bridge-tax cautionary tales (react-native, Electron).
- **HATS is existing evidence** that the declarative render layer retargets: it already interprets one tree description into SVG and Canvas; a native scene graph is a third interpreter, not a rewrite.
- **Projection-principle fit**: a native window is another vessel over the same typed core — peer of Calypso, the CLI, and the hardware vessels. Nothing about the store of record changes.
- **Part III tie-in**: UI is one of the few systems domains where the lawful literature is *ahead* of practice — Elm's architecture is a Moore-machine fold, Conal Elliott's FRP is denotational and lawful, Phil Freeman's comonadic UIs pair component comonads with update monads. SwiftUI is declarative but law-free. The evaluation's sharp question is the wager's question: is there a lawful structure that AppKit's delegate soup erases, and does going native *reduce* the complexity budget or merely relocate the web's share into a bridge?
- **Prior art to mine**: SwiftUI, monomer and gi-gtk-declarative (Haskell native), iced (Rust, Elm-arch), reflex/Fran (FRP lineage), Halogen itself as the incumbent.

Part II — Native services: the analysis turned around

Part I asked "what do we take from existing infrastructure?" Part II asks the reverse: **what services would exist in a world where the substrate already does?** Scored 2026-07-18 from a bluesky-then-sift pass.

The generator: run the degradation clause backwards

Every classic Unix service exists to *compensate for information the substrate throws away*. `grep/sed/awk` exist because types are erased at every boundary — byte-stream repair tools. `make` exists because

justification is erased: the filesystem doesn't know `foo.o` is a consequence of `foo.c`, so a separate tool re-derives it from timestamps. `man` exists because the contract is erased from the binary; `errno` erases cause; logs erase structure; cron erases dependency. So for each service, ask what information was discarded that forced the tool into existence — the substrate-native version is usually the same service *with the information kept*.

Unix answers "what is" (`ps`, `ls`, `cat`). A substrate that keeps types and justifications can answer two questions with no Unix ancestor:

- **why <thing>** — walk the justification chain: why does this file exist, why is this service running, why is this belief held.
- **still? <thing>** — is it still justified, or has a premise died?

These two are the JTMS made into an operating-system service.

The service enumeration

#	Service	Unix ancestor (what it erased)	Status (2026-07-18)	Notes
N1	Provenance / truth maintenance — every derived artifact (built output, generated index, calibration table, plan conclusion) is a belief with a justification; staleness is a system-visible property; retraction propagates	make + cron + memory (erased justification)	PLANNED (demo: jtms-make)	The flagship. Build Systems à la Carte generalized past the build tool; Nix's derivation insight made <i>live</i> via JTMS retraction. Hand-rolled evidence: DeepStar <code>verify</code> (identity/staleness checks = a poll-based TMS for one rig), <code>make api-index</code> staleness, output/-vs-src drift. Seed exists: purescript-jtms + kibitzer as consumer. Prior art to mine: Unison (content-addressed typed code). Design constraints learned from jtms-make (2026-07-18): (a) <i>positivity</i> — service rule-sets must avoid negation-as-failure (a NAF misfire is unretractable in a monotone store); the pattern is positive rules over total observation snapshots (every path axiomatically <code>Exists</code> or <code>Missing</code>), which makes the layer confluent and order-independent. Totality of observation is N1's soundness lever. (b) <i>scaling</i> — the demo's <code>still?</code> is rebuild-and-resaturate per snapshot, fine at 10 ² beliefs; filesystem-scale N1 needs incremental maintenance (retraction-by-replay / semi-naive,

#	Service	Unix ancestor (what it erased)	Status (2026-07-18)	Notes
				already on the jtms roadmap) before it can watch a real artifact graph
N2	Typed events at every joint — observation as a property of the carrier, not the app; tap the codec at any B2/B5/B6 joint and get typed events for free; the event store is a database (DuckDB); tracing = the event DAG; the observability surface is Hylograph over the store	syslog + printf + tcpdump (erased structure)	BLUESKY	Hand-rolled evidence: DeepStar metrology, Chair (the prototype lens), scattered per-project logs. Deployable joint-by-joint
N3	The solver service — Unix has <i>sort</i> ; the substrate has <i>satisfy</i> . Constraint problems (port allocation, package resolution, ES-9 bus routing, schedule packing) expressed in a typed eDSL, dispatched to a resident solver (Z3) behind a carrier	— (no ancestor; solved by hand)	PLANNED	The queued Z3 work is this service's seed. Hand-rolled evidence: every ad-hoc allocation decision in the registry, rig routing done by inspection
N4	The coordination DAG — cron, make, CI, the todo list, and Marginalia's dependency edges are one object: a DAG of typed computations with schedulers and executors à la carte (machine, Claude,	cron + make + CI + todo (erased dependency and premise)	EXISTS (seed)	ShapedSteer is this, per its design. Marginalia's status lifecycle is the hand-rolled shadow. Bosun is a planner with a very short horizon — supervision and planning unify at the limit

#	Service	Unix ancestor (what it erased)	Status (2026-07-18)	Notes
	human). The JTMS supplies what no build system has: plans have premises — "we designed X assuming Y" is a justification, and when Y dies everything resting on it is flagged, not silently rotten			
N5	The typed wireshark — the API index knows where every contract module lives, so a universal wire-tap can decode any substrate socket into values, not hex	tcpdump (erased types)	BLUESKY	Cheap, high joy; near-term buildable off B2 + A4a
N6	Capability audit for agent code — in-family, "what can this program touch" is statically visible in its monad stack; the substrate bounds a generated script's blast radius from its <i>type</i> before anything runs	sudo + hope (erased capability)	BLUESKY	Aimed at how this substrate actually grows: a large fraction of incoming code is Claude-authored. Adversarial security-at-large is deliberately deferred — single-tenant substrate
N7	Resource leases — resources (ports, MIDI endpoints, the ES-9, SD cards, API budgets) as typed, leased values with bracketed acquisition; pre-flight checks become standing assertions in	lockfiles + ls of (erased ownership)	BLUESKY	Seeds exist: the port registry, Bosun's Exposure, DeepStar pre-flight

#	Service	Unix ancestor (what it erased)	Status (2026-07-18)	Notes
	the TMS rather than a poll at showtime			
N8	Coreutils dissolution — once C3 pipes carry values, the standard library <i>is</i> coreutils: grep <i>is</i> filter, cut <i>is</i> a row-type projection, sort <i>is</i> sortBy, join <i>is</i> Map. Surviving "commands" are published typed functions; the -o json/--porcelain flag-jungle dies because presentation is chosen by the consumer of values, not the producer	coreutils (erased types)	BLUESKY	Gated on C1 + C3. Prior art and cautionary tales: PowerShell, nushell (objects in pipes; weak types, closed ecosystems)
N9	Protocol extensions — session types (protocol state machines in types: out-of-order messages don't compile); protocol upgrade as compiled migration ($v_n \rightarrow v_{n+1}$) a total function, so "will old clients break" is a type error)	—	DEFERRED	Heavy machinery relative to current pain; revisit when B2 extraction exists

The combinator vocabulary (corollary to N8)

The pipe begat byte-stream specialists (sort, grep, uniq, sed); the typed pipe begets **effect-pattern specialists**. The stdlib already covers list-shaped work — filter *is* grep, **sortBy** *is* sort. What the substrate adds is the vocabulary for the boundary-crossing patterns every ops task repeats, most of which carry Part III laws:

- **gather** — concurrent traversal accumulating failures instead of short-circuiting: $f\ a \rightarrow (a \rightarrow \text{Aff}\ (\text{Either}\ e\ b)) \rightarrow \text{Aff}\ \{\text{oks}, \text{fails}\}$. The typed `for x; do cmd || note-failure`; Validation semantics lifted to effects.
- **reconcile** — desired-vs-actual: two $\text{Map}\ k\ v$ in, `{ create, change, delete }` plan out. THE ops combinator (Kubernetes controllers, Bosun group management, archive sync are all this one function); where the edit-lens/change-structure import lands.
- **converge** — effectful step to fixpoint: $\text{Eq}\ s \Rightarrow (s \rightarrow \text{Aff}\ s) \rightarrow s \rightarrow \text{Aff}\ s$. Saturation, retry-until-stable, eventual consistency; monotone step on a finite lattice $\Rightarrow$ termination for free (Part III).
- **groupFold** — $(a \rightarrow k) \rightarrow (f\ a \rightarrow b) \rightarrow f\ a \rightarrow \text{Map}\ k\ b$: the `awk / GROUP-BY` idiom as one combinator.
- **Bracketed traversals** — "for each card/repo/service: acquire, act, release", release guaranteed (N7's leases as a HOF).
- **Receipted folds** — folds that keep their why (Writer-monoid pattern): N1 in the small; any script can answer *why* about its own output.
- **Ambient traverseOf** — effectful optics as the typed `find -exec / chmod -R`: one Traversal, any Applicative (exists in the optics libs; the work is making it shell-ambient).

The claim to hold N8 to: most everyday ops scripts are compositions of five or six of these. When C1/C3 land, this suite is what ships in the prelude of the shell.

The sift criteria

1. **Pain evidence** — do we already hand-roll it? (The market test, same as Part I's extraction rule.)
2. **Substrate leverage** — does it require the lingua franca, or could plain Unix do it? If Unix could, it's not a substrate service.
3. **Incremental deployability** — can it start weak with clear errors (the Quartermaster bar), coexisting via the degradation clause?

Ranked by these: $N1 > N2 > N3 > N4$ as commitments; N5/N6/N7 warm; N8 gated on Part I's C1+C3; N9 deferred.

The convergence — spec for the JTMS/Z3 work

Running the analysis in the *opposite* direction from Part I landed on JTMS and Z3 — the exact two pieces of work already queued. That convergence is the plan's strongest signal, and it upgrades the framing of that work: **not "finish the library" but "build the first native services" (N1 and N3)**, carrier-fronted and typed, linked to ShapedSteer (N4) as their largest consumer. Concrete first tenants:

- kibitzer as N1's existence proof (already the JTMS consumer);
- DeepStar's checks re-expressed as standing justifications (N1 + N7): "Link is on", "ES-9 in Hosted mode", "running code matches source" as maintained beliefs, not polls;
- ES-9 bus routing or port allocation as N3's first solver query;
- ShapedSteer plans carrying premises, invalidated by N1 retraction.

The subsumption target: spreadsheets, notebooks, build systems

Called out explicitly because it is core to the ShapedSteer idea: the kinds of tasks people use **spreadsheets, notebooks, and makefiles/build systems** for, we are explicitly going to try to subsume.

Each of those tools is a DAG-of-computations surface that erases something essential:

- the **spreadsheet** erases the program — the grid is the code, formulas hide in cells, there is no whole to read, type, or test;
- the **notebook** erases execution order and dependency — hidden state, out-of-order cells, no honest "still valid?";
- the **build system** erases types — bytes and timestamps at every edge.

ShapedSteer keeps all three: typed nodes, explicit dependencies, rebuild semantics à la carte, mixed executors. It subsumes the *tasks*, not the file formats.

A sharpening from pointing jtms-make at the real afc-work Makefile (2026-07-18): in the wild, "the Makefile" conflates **building** with **provisioning and operating** — and the subsumption splits accordingly. Of ~20 top-level targets, one is a dependency edge; the rest are docker-up/down (Bosun's job, already done better by a typed compose spec), a **check-tools** probe (a four-line hand-rolled Quartermaster), delegating façades, and **help** — the verb menu, which exists in every repo on earth because Unix has no typed registry of a project's operations and make is the accidental universal verb dispatcher. So: build semantics → ShapedSteer/N1; service run-state → Bosun; environment probes → Quartermaster (E4); the menu role → a typed operations surface (Chair; eventually C1/C3). The phony-fraction of a Makefile is a litmus for which document you're actually holding. Prize specimen: **api-index** is the file's one genuinely build-shaped task, yet written phony because make cannot express its true premise set ("every .purs file under twenty repos") — the staleness information was erased by the tool's weakness, not by choice. N1 has no such limit; **still? API-INDEX.md** is the exact question we currently answer by remembering.

The projection principle

The deeper claim under the subsumption target: **all of this is completely expressible in code**, with spreadsheets, notebooks, and sankey/flow diagrams being *informational and sometimes ergonomic projections of that code* — views, never the store of record.

The music stack has already proven this pattern at full scale: music theory (Harmonia) and the tidal DSL are unified into one semantic core, then served in different UI vessels for different purposes — Calypso for live performance, the CLI for sessions, sketches for exploration, hardware realisers for sound. Nobody mistakes Calypso's panes for the music.

Applied to N4: **ShapedSteer is a candidate universal DSL for the spreadsheet/notebook/build-system domains** — one typed core (the DAG of computations), with the grid view as its spreadsheet projection, the notebook view as its notebook projection, timeline and sankey as its flow projections. Hylograph exists precisely to make such projections cheap. This kills each subsumed tool's fatal flaw in one move: the grid is a *view* of the program instead of being the program. And the DSL consumes the native services — N1 for staleness and provenance of every cell, N3 for solver-backed cells, N2 for execution observability.

The principle is now proven twice (music; jtms-make's data-flow and belief-chain views over one KB), and jtms-make added a corollary: **projections double as instruments**. Rendering a document as a projection of a belief store doesn't just display it — it *measures* it (the phony-fraction litmus, the five undeclared phonies, the provisioning/building split all fell out of the render). Expect each native service's projections to earn their keep as audits of whatever they project. Method note, also twice-proven: build the semantic core headless and tested before any pixels — the projections then cost days, not weeks.

Part III — The categorical lens: laws in the substrate

Two questions (2026-07-18): which common typeclasses have useful, lawful instances hiding in operating-system artifacts? And which abstractions would serve the substrate that mathematics has not yet delivered into Haskell/PureScript?

Motivating precedent: the systems $\rightarrow$ mathematics $\rightarrow$ typeclass pipeline is real and recent — **Selective functors (2019) were imported into Haskell from build systems** (Mokhov, of Build Systems à la Carte). A substrate project written in the lingua franca is positioned to drive more such imports. The test for any candidate: does the artifact obey laws worth property-checking? **An instance is a compressed specification; its laws are free tests** — the cheap first rung of the verification ladder (per the satisfice stance).

Instances already present in substrate artifacts

Artifact	Structure	What the laws buy
Make's assignment flavors ($=/:=$, $?=$, $+=$)	Last / First / append monoids per variable	the config-layering semantics fight, named; jtms-make's parser touches this daily
Config layering generally (defaults $\langle \rangle$ system $\langle \rangle$ user $\langle \rangle$ flags), env composition	monoid fold over right-biased maps	associativity = "layering order composes"; explicit choice of monoid per key
JTMS saturation	closure operator on the claim-set lattice (Knaster-Tarski lfp of the rule-consequence operator)	extensive, monotone, idempotent — three QuickCheck properties the engine must pass forever
Harmonia's quantiser	Galois adjoint of scale inclusion (as $\text{floor} \dashv \mathbb{Z} \hookrightarrow \mathbb{R}$)	adjunction laws = quantisation correctness, already latent in Quantise tests
Paths	free monoid on segments; lenses into the FS tree; $\text{cwd} = \text{zipper} / \text{Store-comonad focus}$	get-put/put-get for anything path-addressed (C2's AssetLocation should be optic-valued)
<code>du / chmod -R / find</code>	one Traversable (the tree), many Applicatives (Const (Sum Bytes), Effect)	coreutils dissolution (N8), categorically stated
Typed pipelines (C3)	Kleisli composition (Category laws)	associativity = pipelines refactor freely
Event sinks & routing (N2)	Contravariant loggers; Divisible fan-out; Decidable routing on sums	event plumbing gets a complete, boring algebra

Artifact	Structure	What the laws buy
es9/fh2 SysEx configs	lenses/isos over device state	the round-trip parse/print tests already run ARE the iso laws, unnamed
Overlay FS / OCI layers	monoid of layers acting on filesystem states	layer reordering/collapse reasoning
Multi-host state (registries, Bosun views)	join- semilattices (CRDTs)	commutative-idempotent merge = sync without coordination
Build doctrines	Applicative = make, Monad = Shake, Selective = Dune	ShapedSteer's rebuild-à-la-carte inherits the classification; Selective's dependency under/over-approximation = the "read-set extraction" the jtms sketch wants
Plans with mixed executors (N4)	free structures interpreted by natural transformations (machine / Claude / human executors = interpreters)	executor-independence stated as a theorem shape

Corollary, and another instrument: **where there's no lawful instance, there's a wart**. Pipes compose; signals don't. Paths concatenate; errno doesn't. Unix's good parts are accidentally good algebra, and the instance-hunt doubles as a design audit.

Wanted imports (mathematics → the lingua franca), ranked

- 1. Order theory as first-class citizens** — **Lattice**, **ClosureOperator**, **GaloisConnection** classes with laws. Thin-to-absent in PureScript, yet the native services are mostly order-theoretic, not algebraic: N1 saturation, permission lattices (N6), version ranges, CRDT joins, abstract interpretation (Minard's views as $\alpha \mapsto \gamma$), the quantiser. Cheapest import, broadest coverage.
- 2. Change structures / incremental semantics** — incremental λ -calculus; DBSP's algebraic form (abelian groups + delay). This is the N1 scaling boundary already named in Part II: rebuild-vs-retraction is the question "*what is the derivative of **saturate**?*". Import shape: a **Change** a class with **diff/apply** laws; prize: rules compiled to their own deltas.
- 3. Edit/delta lenses** (bidirectional-systems literature — Pierce/Hofmann lineage): propagate *edits*, not whole states. Sync is edit-lens shaped: the mutual backup, SysEx deltas to hardware, the personal archive. Rich literature, near-zero FP presence.
- 4. Adopt Selective** (imported 2019, unused here) for ShapedSteer plan analyzability — static read-sets from dynamic-ish plans.
- Bluesky tier: **sheaf conditions** for multi-host consistency (local views agreeing on overlaps glue to a global view — the Marginalia/chair-server "not synced" seam is a sheaf-condition failure with a name); **linearity** for resource leases (N7 — imported into Rust but not the lingua franca, which is partly *why* the substrate leans on Rust at the edges); **coalgebra / bisimulation** as the right equivalence for services (DeepStar's **verify** is bisimulation-flavored).

Scoring note: items here graduate by acquiring (a) a lawful PureScript encoding and (b) a substrate tenant. Order theory + N1 is the natural first pairing — the closure-operator laws could join the jtms testkit almost immediately.

Mark-to-market protocol

Applies to all parts — Part I carriers, Part II services, and Part III instances/imports are scored in the same pass.

1. **Cadence:** re-score at natural junctures — after any project that touches a carrier, or roughly quarterly, whichever comes first. Each re-scoring: regenerate the API index (**make api-index**), re-run the evidence greps implied by each row, update statuses and the ledger below. The scoring session is cheap (< 1 session) by design.
2. **The extraction rule: AD HOC ×3 or more ⇒ extraction is scheduled**, not merely noted. Three independent hand-rollings of the same joint are the market saying the part exists and wants a name. Current extractions owed by this rule: B2 (protocol pattern, ×6), B4 (process invocation, ×3), B5 (control-socket library, ×3), D1 (embedded-DB shape, ×3).
3. **The jointing rule:** new projects joint against this enumeration. Design conversations should be able to say "this boundary is a B5, use the carrier" — and where nothing fits, amend the document first.
4. **The demotion rule:** statuses can go down. If a part rots or its only instance is abandoned, say so; the document is a market, not a trophy case.
5. **Sharper-tools clause:** where a carrier is load-bearing and small (B3's error algebra, C3's pipeline laws), consider per-artifact verification (property tests first; Lean/TLA+ where correctness is the risk) per the standing satisfice-not-religion guidance.

Ledger

Date	Scored by	Movements
2026-07-17	Claude + AFC	Initial enumeration. 4 extraction debts identified (B2, B4, B5, D1). Named gaps: B3, C1, C3, E4.
2026-07-18	AFC review + Claude	Openness clause added (crux, clause 3; motivated at D3). E4 MISSING → PLANNED (Quartermaster #238; "Nix until we can rewrite Nix"; PureNix to assess). B5 widened to daemon-creation standardization + the foreign-language-daemon question. B6 marked as key technology (HTTPurple = default in-family service carrier). C2 widened from paths to location addressing (pathy to evaluate; URI-general AssetLocation ADT sketched). D2 direction set: rowtype-yoga compile-time schema-validated SQL + minard-db integration. E2 gains the BEAM-as-container question and Bosun visibility extension. Music over-indexing acknowledged as evidence-base skew, expected to rebalance.

Date	Scored by	Movements
2026-07-18	AFC + Claude	Part II added — native services from the turned-around analysis (bluesky → sift, nothing culled). N1 provenance/TMS and N3 solver committed as PLANNED; N2 typed events and N4 coordination DAG (ShapedSteer seed) anchored; N5–N8 BLUESKY; N9 deferred. Convergence recorded: the queued JTMS/Z3 work = N1 + N3, reframed from "finish the library" to "build the first native services", with ShapedSteer as largest consumer. Subsumption target declared (spreadsheets, notebooks, build systems) and the projection principle stated: code is the store of record, UI vessels are projections — music stack (Harmonia + tidal core, Calypso/CLI/hardware vessels) cited as the proven instance; ShapedSteer positioned as candidate universal DSL for the subsumed domains.
2026-07-18	Claude	N1 has a demo: <code>j tms-make</code> (purescript-hylograph-demos) — a Makefile's semantics as monotone JTMS rules, staleness as derived belief with provenance, the build DAG as a Sankey projection of the derivation DAG; ships with the afc-work root Makefile as a scenario (the third subsumption-target round-trip, self-hosting). Notable: the engine surfaced five targets never declared .PHONY that make silently rebuilds every run — the provenance layer reading the Makefile more honestly than its author.
2026-07-18	AFC + Claude	Subsumption section sharpened from the demo's reading of the real Makefile: wild Makefiles conflate building with provisioning/operating — build → ShapedSteer/N1, run-state → Bosun, probes → Quartermaster, verb menu → typed ops surface. Phony-fraction as document litmus; <code>api-index</code> recorded as the specimen of tool-forced staleness erasure.
2026-07-18	AFC + Claude	Three additions from review: crux clause 4 (parse-don't-validate + MISU plumbed to the bottom — totality-of-snapshot cited as the exemplar; the <code>set -euo pipefail</code> genre as what it replaces); the complexity wager (typed-lingua-franca additions to the complexity budget must be paid back by typeclasses and laws — every Unix sublanguage accreted is a credit waiting); the combinator vocabulary as N8's corollary (gather, reconcile, converge, groupFold, bracketed traversals, receipted folds, ambient traverseOf — effect-pattern specialists, the typed pipe's sort grep uniq). Also booked: a tutorial session unpacking Part III's mathematics.

Date	Scored by	Movements
2026-07-18	AFC + Claude	Part III added — the categorical lens. Instances found in artifacts (Make's assignment flavors as monoids; saturation as closure operator with three free laws; the quantiser as Galois adjoint; paths as free-monoid + lenses; SysEx round-trips as unnamed iso laws; CRDT semilattices; the Applicative/Monad/Selective build classification). Corollary: no lawful instance $\Rightarrow$ a wart (signals, errno) — the lens is another audit instrument. Wanted imports ranked: order theory first-class (top; pairs with N1 immediately), change structures/DBSP (= the N1 scaling question), edit lenses (sync-shaped), adopt Selective for ShapedSteer; bluesky: sheaf conditions for multi-host consistency, linearity for leases, bisimulation for service equivalence. Precedent noted: Selective functors were themselves a systems$\rightarrow$typeclass import (2019).
2026-07-18	AFC + Claude	Build reflections folded back. N1 gains two learned design constraints: positivity (no NAF; positive rules over total observation snapshots — totality is the soundness lever) and the scaling boundary (rebuild-and-resaturate fine at 10^2 beliefs; filesystem scale needs retraction-by-replay/semi-naive). Projection principle: proven twice; corollary added — projections double as instruments (rendering a belief store measures the projected document); method note: headless tested core before pixels. Demo committed (demos repo de131dc), archived private (github.com/afcondon/jtms-make), tracked as Marginalia 255 under purescript-hylograph-demos (150).
2026-07-18	AFC	F4 added (BLUESKY) — the native-vessel question, captured mid-stream: a project to evaluate interface-building technologies (AppKit/SwiftUI vs web frameworks vs Halogen) toward a fully-FP, fully-native UI library as a substrate tenant. F1's SETTLED explicitly scoped to the web vessel. Sharp question is the wager's: is there a lawful structure AppKit's delegate soup erases, and does going native reduce the complexity budget or relocate it into a bridge? Categorical UI literature (Elm-as-Moore-fold, Elliott FRP, Freeman comonadic UIs) noted as unusually ahead of practice — a Part III wanted-import candidate.